

Collaboration is prohibited by CUNY bylaws on Plagiarism.

Student Name: Jiaxing Kong

ID: 24075114

- 1) A 0/1 knapsack can hold a maximum weight of $Weight=10$
 Suppose that there are 5 items, with corresponding weights $w = \{5, 4, 2, 3, 1\}$ and corresponding values $v = \{10, 40, 60, 50, 70\}$.

Find the maximum value the knapsack can hold from the 5 items while the total weight of the items is at most 10. Moreover, you must determine which items will be selected using Dynamic Programming.

You must show the table with 6 rows labeled 0, 1, 2, 3, 4, 5 corresponding to the items (first row no items), and 11 columns labeled 0, 1, 2, ..., 10 as explained in the distance learning lecture on Dynamic Programming.

i	1	2	3	4	5
w	5	4	2	3	1
v	10	40	60	50	70

$MaxVal[i][w]$ = maximum value using the previous i th items with maximum weight w

$$MaxVal[i][w] = \begin{cases} MaxVal[i-1][w], & \text{if adding item } i > \text{max weight} \\ \max(MaxVal[i-1][w], v_i + MaxVal[i-1][w-w_i]) \end{cases}$$

MaxVal:

i \ w	0	1	2	3	4	5	6	7	8	9	10
0	0	0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	10	10	10	10	10	10
2	0	0	0	0	40	40	40	40	40	50	50
3	0	0	60	60	60	60	100	100	100	100	100
4	0	0	60	60	60	110	110	110	110	150	150
5	0	70	70	130	130	130	180	180	180	180	220

$$MaxVal[5][10] = 220$$

$$\begin{aligned} MaxVal[5][10] &= MaxVal[4][9] + 70 = MaxVal[3][6] + 50 + 70 \\ &= MaxVal[2][4] + 60 + 50 + 70 = MaxVal[1][0] + 40 + 60 + 50 + 70 \\ &= MaxVal[0][0] + 40 + 60 + 50 + 70 \end{aligned}$$

Therefore, item 2, 3, 4, 5 are selected

Collaboration is prohibited by CUNY bylaws on Plagiarism.

Student Name: Jiaying Rong

ID: 24075114

- 2) Find the total number of sequences of length n (using 0 and 1) such that no two 0s are next to each other. For example, for $n = 2$, we have 3 possible sequences: 01, 10, 11. Let $f(n)$ be the number of such sequences. Let's call these sequences "NO-00 sequences." We have $f(1) = 2, f(2) = 3$.

a) Find a recursive relation for $f(n)$ in turn of previous values of n and explain your answer.

$$f(n) = f(n-2) + f(n-1) \text{ when } n \geq 3$$

① When the sequence ends with 0, because NO-00 sequences, the sequence must ends with 10. In this case, we only need to find the number of NO-00 sequences of length $n-2$, which is $f(n-2)$

② When the sequence ends with 1, we only need to find the number of NO-00 sequences of length $n-1$, which is $f(n-1)$

③ These 2 cases includes all cases of NO-00 sequences of length n

- b) Write a Dynamic Programming program to solve for $f(n)$ and determine the computational complexity.

```
int f(int n) {
    if (n == 1 || n == 2) return n + 1;
    int result[n + 1];
    result[1] = 2;
    result[2] = 3;
    for (int i = 3; i <= n; i++) {
        result[i] = result[i - 2] + result[i - 1];
    }
    return result[n];
}
```

Time Complexity: $O(n)$

Space Complexity: $O(n)$

- c) Write a recursive program (direct, simple) and determine the computational complexity.

```
int f(int n) {
    if (n == 1 || n == 2) return n + 1;
    return f(n - 2) + f(n - 1);
}
```

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

Collaboration is prohibited by CUNY bylaws on Plagiarism.

Student Name: Jiaxing Kong

ID: 24075114

3) Consider an infinite sequence $a(n)$ for $n = 0, 1, 2, \dots$, the sequence $[1, 3, 2, -1, -3, -2, 1, 3, \dots]$

a) Write $a(n)$ as a recursive function (direct, simple), in terms of the previous term of the previous terms.

```
int a(int n)
{
    if (n == 0) return 1;
    if (n == 1) return 3;
    return a(n-1) - a(n-2);
}
```

b) Write a Dynamic Programming procedure to calculate $a(n)$ using the recursive function determined in 3-a.

int Calc (int n)

```
{
    if (n == 0) return 1;
    if (n == 1) return 3;
    int result[n+1];
    result[0] = 1;
    result[1] = 3;
}
```

```
for (int i = 2; i <= n; i++) {
    result[i] = result[i-1] - result[i-2];
}
return result[n];
```

c) Which of the two methods has better time-complexity/ Explain

Dynamic Programming method has better time-complexity with $O(n)$ over $O(2^n)$ recursive method: $a[n]$
 $a[n-1] \quad a[n-2]$
 $a[n-2] \quad a[n-3] \quad a[n-3] \quad a[n-4]$ which is a binary tree with height n

d) Can you find a formula to determine $a(n)$?

$$a(n) = \begin{cases} 1, & \text{if } n \bmod 6 = 0 \\ 3, & \text{if } n \bmod 6 = 1 \\ 2, & \text{if } n \bmod 6 = 2 \\ -1, & \text{if } n \bmod 6 = 3 \\ -3, & \text{if } n \bmod 6 = 4 \\ -2, & \text{if } n \bmod 6 = 5 \end{cases}$$

Collaboration is prohibited by CUNY bylaws on Plagiarism.

Student Name: Jiaying Rong

ID: 24075114

4) Consider an infinite sequence $[1, 2, 4, 10, 42, 422, \dots]$, where $a[0]=1$ and $a[1]=2$,

a) Write $a(n)$ as a recursive function with respect to the previous terms for $n \geq 2$.

$$a(n) = a(n-2) * a(n-1) + 2, (n \geq 2)$$

b) Write a recursive algorithm to calculate $a[n]$ give an idea of the worst-case.

```
int a(int n) {
    if (n == 0 || n == 1) return n + 1;
    return a(n-2) * a(n-1) + 2;
}
```

Worst Case:

Time Complexity: $O(2^n)$

Space Complexity: $O(n)$

c) Write a Dynamic Programming algorithm and give an idea of the worst-case.

```
int a(int n) {
    if (n == 0 || n == 1) return n + 1;
    int result[n + 1];
    result[0] = 1;
    result[1] = 2;
    for (int i = 2; i <= n; i++) {
        result[i] = result[i-2] * result[i-1] + 2;
    }
    return result[n];
}
```

Worst Case:

Time: $O(n)$

Space: $O(n)$